CS 201, Fall 2016 — Oct 28th — Exam 2 Name: _____

Question 1. [5 points] Consider the following static method:

```
public static int mystery(int[] arr) {
    int x = 0;
    for (int i = 0; i < arr.length; i++) {
        x += arr[i];
    }
    for (int i = 0; i < arr.length; i++) {
        x += arr[i];
    }
    return x;
}
```

Count how many times each loop runs
-First loop: Goes through all N elements → N
-Second loop: Goes through all N elements again → N

Total number of iterations: N + N = 2N

In Big-O notation, we can ignore constant numbers like 2.

Therefore, O(N)

Let N be the number of elements in the array `arr`. State a worst-case big-O upper bound on the running time of this method in terms of N. Explain your answer briefly.

Answer: O(N)

Question 2. [5 points] Consider the following static method:

```
public static int numNegativeRows(int[][] matrix) {
    int count = 0;
    for (int i = 0; i < matrix.length; i++) {
        int rowSum = 0;
        for (int j = 0; j < matrix[i].length; j++) {
            rowSum += matrix[i][j];
        }
        if (rowSum < 0) { count++; }
    }
    return count;
}
```

Assume that the array `matrix` is square (has the same number of rows and columns), and that N is the number of rows/columns. State a worst-case big-O upper bound on the running time of this method in terms of N. Explain your answer briefly.

Answer: $O(N^2)$

The outer loop runs N times for the rows, and the inner loop runs N times for the columns.

Since the loops are nested, we multiply N × N = N^2.

Therefore, the running time is $O(N^2)$.

Question 3. [5 points] State a big-O upper bound on the worst-case running time of the following method, where the problem size N is the number of rows/cols in the array `mat`. You may assume the array has the same number of rows as columns. Briefly justify your answer, by indicating the approximate *average* number of iterations for each loop.

```
public static double diagSum(double[][] mat) {
  double sum = 0.0;
  int N = mat.length;
  for (int i = 1; i < N; i *= 2) {
    sum += mat[i][i];
  }
  return sum;
}
```

Answer: O(log N)

Important: Notice `i *= 2`.

This means that i doubles after every iteration, instead of increasing by 1.

Therefore, the loop runs approximately: log_2 N

We can ignore the base in Big-O, so the answer is just O(log N)

Question 4. [5 points] Consider the following static method:

```
public static boolean isUpperTriangular(double[][] matrix) {
    for (int i = 0; i < matrix.length; i++) {
        for (int j = 0; j < i; j++) {
            if (matrix[i][j] != 0.0) {
                return false;
            }
        }
    }
    return true;
}
```

Assume that the array `matrix` is square (has the same number of rows and columns), and that N is the number of rows/columns. State a worst-case big-O upper bound on the running time of this method in terms of N. Explain your answer briefly.

Answer: $O(N^2)$

In the worst case, all the numbers below the diagonal are zero, so the method must check every one of them.

The total number of iterations is 0 + 1 + 2 + ... + (N-1) = N(N-1)/2

Remember the above equation for your exam.

Since Big-O ignores lower order terns, the running time is $O(N^2)$.

Question 5. [5 points] Consider the following static method:

```
public static boolean checkMatrix(double[][] mat) {
    int N = mat.length;
    for (int i = 1; i < N; i *= 2) {
        for (int j = 0; j < i; j++) {
            if (mat[i][j] < 0) {
                return false;
            }
        }
    }
    return true;
}
```

Assume that the array `matrix` is square (has the same number of rows and columns), and that N is the number of rows/columns. State a worst-case big-O upper bound on the running time of this method in terms of N. Explain your answer briefly.

Answer: O(N)

To answer this question, think about three things:

1. Starting value: `i = 1`
2. Stopping condition: `i < N`
3. Update: `i *= 2` (double each time)

Suppose N = 10, write the values:

`1 → 2 → 4 → 8 → 16`

Since 16 is not less than 10, stop at 8.

So total iterations = 1 + 2 + 4 + 8 = 15 < 2N (20)

This means the total is always less than 2N, and remember we say big-O is the upper-bound of running time, so the answer is: O(N)